



# 1-3 탐색알고리즘

# 우리는 왜 탐색(Search)을 할까?



## 시간에 민감한 작업

경로를 이탈했습니다. 내비게이션은 1초라도 빨리 새로운 목적지 경로를 찾아야 합니다.



## 효율성

수십억 개의 웹 페이지 속에서 내가 원하는 정보를 단 몇 초 만에 정확하게 찾아냅니다.

방대한 데이터 속에서 정보를 빠르고 정확하게 찾는 기술, 이것이 바로 탐색 알고리즘입니다.



# 데이터의 지도, '데이터 구조'



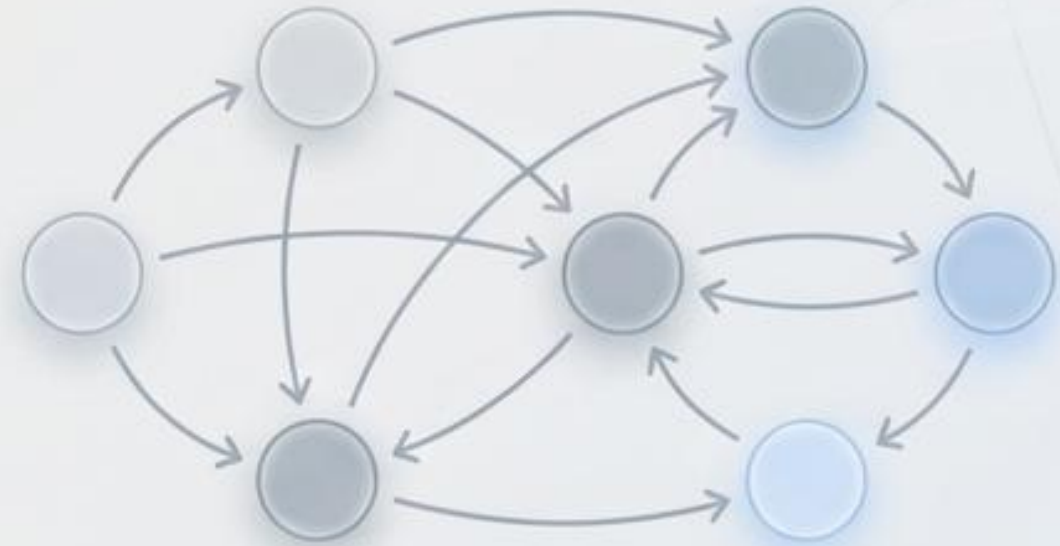
오늘의 목표: 한 줄로 나열된 선형  
데이터 구조 안에서 원하는 값 찾기!

# 선형 vs 비선형 데이터 구조



## 선형 데이터 구조

- ✓ 요소들이 차례로 연결되는 순차적 방식 (1:1 구조)
- ✓ 구현이 쉽고 특정 요소에 직접 접근 가능 (배열 등)
- ✓ 적용 알고리즘: 순차 탐색, 이진 탐색



## 비선형 데이터 구조

- 한 요소가 여러 요소와 연결된 계층/관계형 (1:n, n:n 구조)
- 특정 노드에 한 번에 접근 불가
- 적용 알고리즘: 깊이 우선 탐색, 너비 우선 탐색



## 무식하지만 확실한 방법: 순차 탐색 (Sequential Search)



- 📖 **정의:** 리스트의 첫 번째 요소부터 마지막까지 순서대로 하나씩 확인하며 원하는 값을 찾는 알고리즘.
- 🔍 **특징:** 도서관 책이 뒤죽박죽 섞여 있어도 1번 책부터 끝까지 다 찾아보면 무조건 찾을 수 있습니다.
- ☆ **핵심:** 데이터가 정렬되어 있지 않아도 사용할 수 있습니다.

# 순차 탐색의 작동 원리 (목표: '5' 찾기)



총 8번의 비교 끝에 탐색 성공!



# 순차 탐색의 현실 (수행 시간)



## 최선의 경우 (운이 좋을 때)

- ▶ 찾는 값이 맨 앞에 있을 때  
→ 비교 횟수 1번



## 최악의 경우 (운이 나쁠 때)

- ▶ 찾는 값이 맨 끝에 있거나  
아예 없을 때 → 비교 횟수  $n$ 번

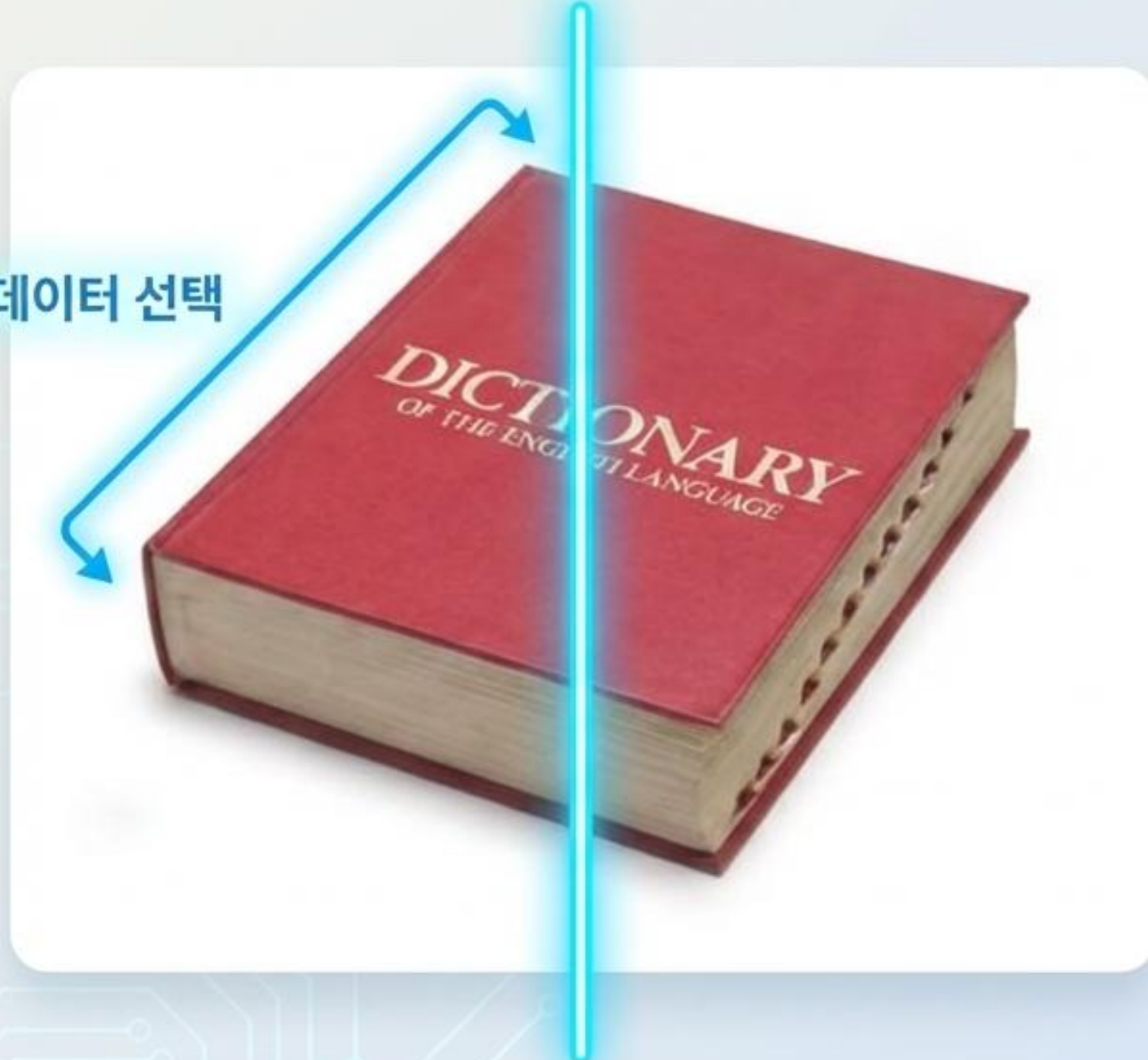


**Takeaway:** 데이터( $n$ )가 많아질수록 탐색 시간도  
정비례하게 알히늘어나는  **$O(n)$** 의 한계를 가집니다.



# 반으로 쪼개는 마법: 이진 탐색 (Binary Search)

50% 데이터 선택



**정의:** 리스트의 중간값을 확인하고, 찾는 값이 중간값보다 작으면 왼쪽 절반, 크면 오른쪽 절반을 탐색하는 과정을 반복하는 알고리즘.

**절대 규칙:** 데이터를 탐색하기 전에 반드시 정렬되어 있어야만 작동합니다.



# 이진 탐색 (Binary Search): 데이터의 바다에서 단숨에 정답을 찾는 법

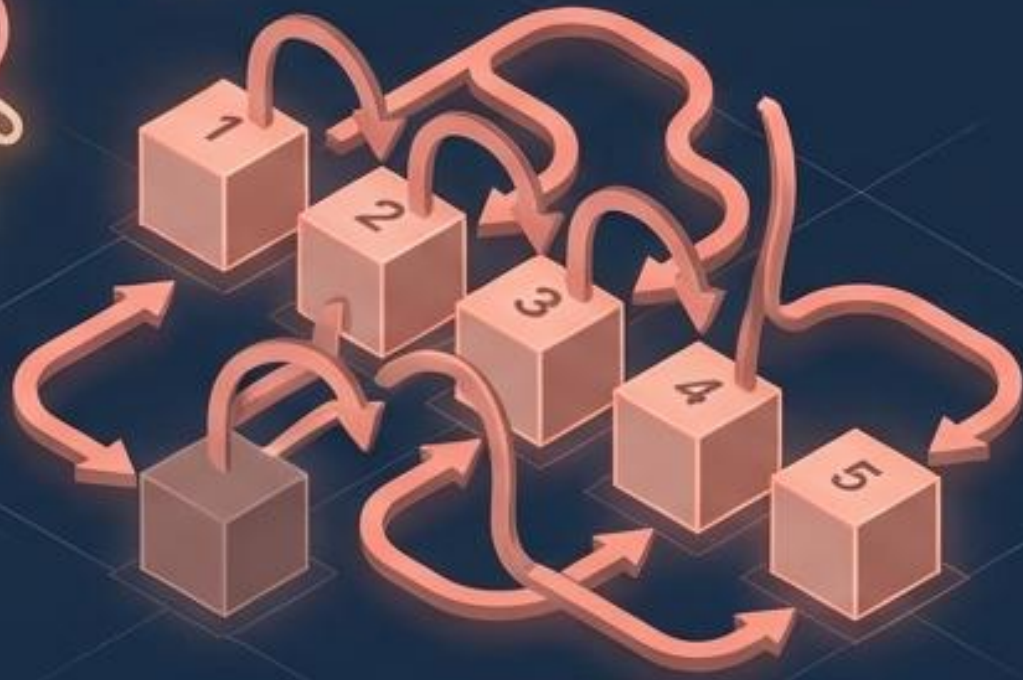


파이썬으로 구현하는 이진 탐색(Binary Search) 알고리즘 완벽 해부

# 1부터 100 사이의 숫자를 가장 빨리 맞히는 방법은?

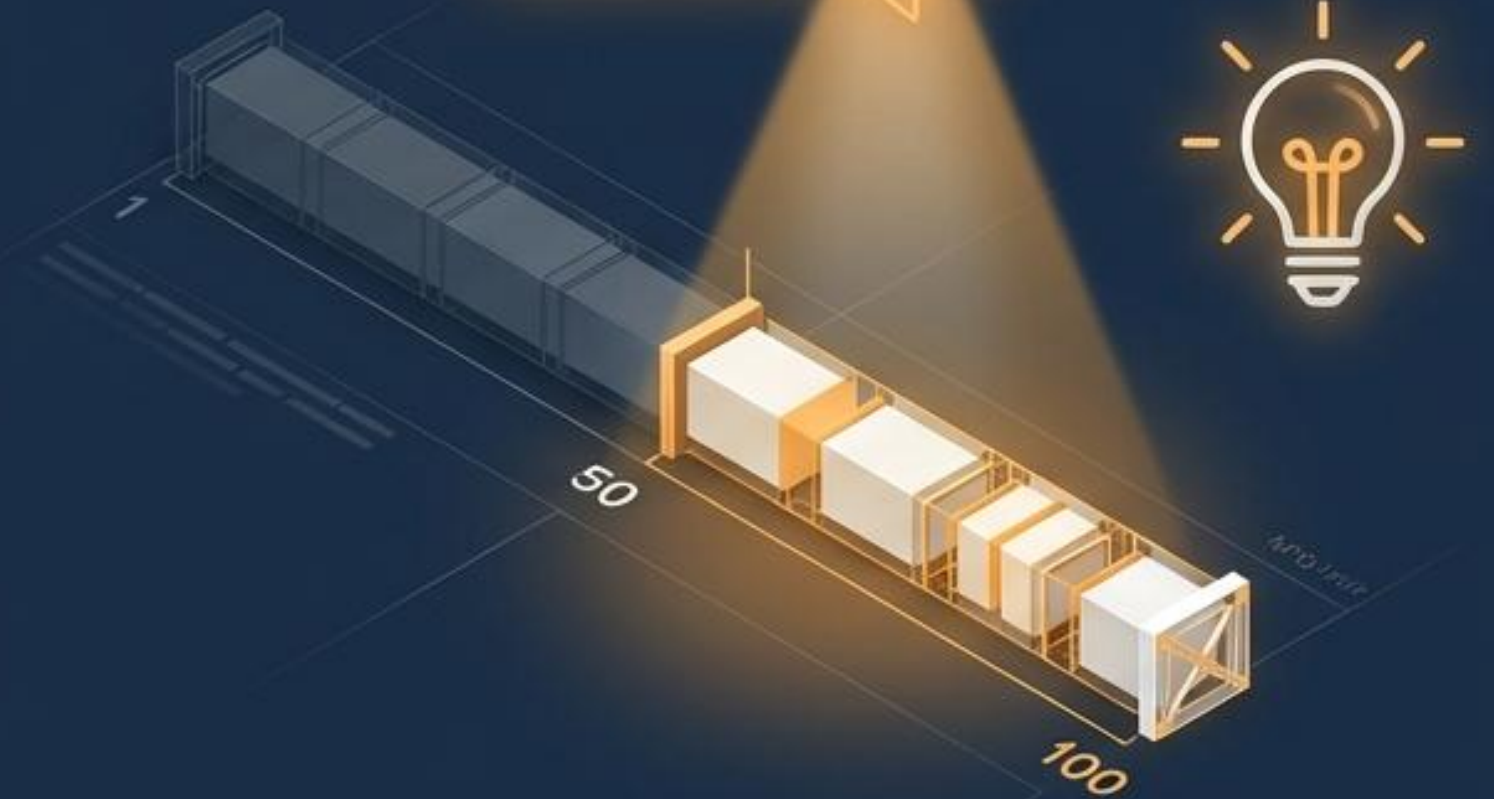
순차 탐색 (바보 같은 방법)

1인가요? 2인가요? 3인가요...



이진 탐색 (스마트한 방법)

50보다 큰가요?



절반을 과감하게 버리는 것, 이것이 효율적인 탐색 알고리즘의 시작입니다.



# 속도의 차이가 알고리즘의 차이를 만듭니다

100만 번의 수고를 20번으로 줄이는 마법. 하지만 이를 위해선 하나의 절대적인 조건이 필요합니다.

| 탐색 방식            | 순차 탐색 (Linear)   | 이진 탐색 (Binary)  |
|------------------|------------------|-----------------|
| 작동 원리            | 처음부터 끝까지 하나씩 확인  | 반씩 쪼개며 탐색 범위 축소 |
| 100만 개 탐색 시 (최악) | 1,000,000번 확인 필요 | 단 20번 확인으로 종료!  |
| 필수 전제 조건         | 없음               | 정렬 (Sort)       |

절반을 과감하게 버리는 것, 이것이 효율적인 탐색 알고리즘의 시작입니다.

# 대전제: "모든 데이터는 정렬되어 있어야 한다"

1. 정렬 전 (Unsorted)



2. `.sort()` 적용 (Sort)



3. 정렬 후 (Sorted)



탐색을 시작하기 전, `.sort()` 함수는 선택이 아닌 필수입니다.  
데이터가 크기순으로 정렬되어 있어야만 컴퓨터가 '어느 방향으로 갈지'를 결정할 수 있습니다.



# 범위를 좁히는 3개의 지시자 (Pointers)

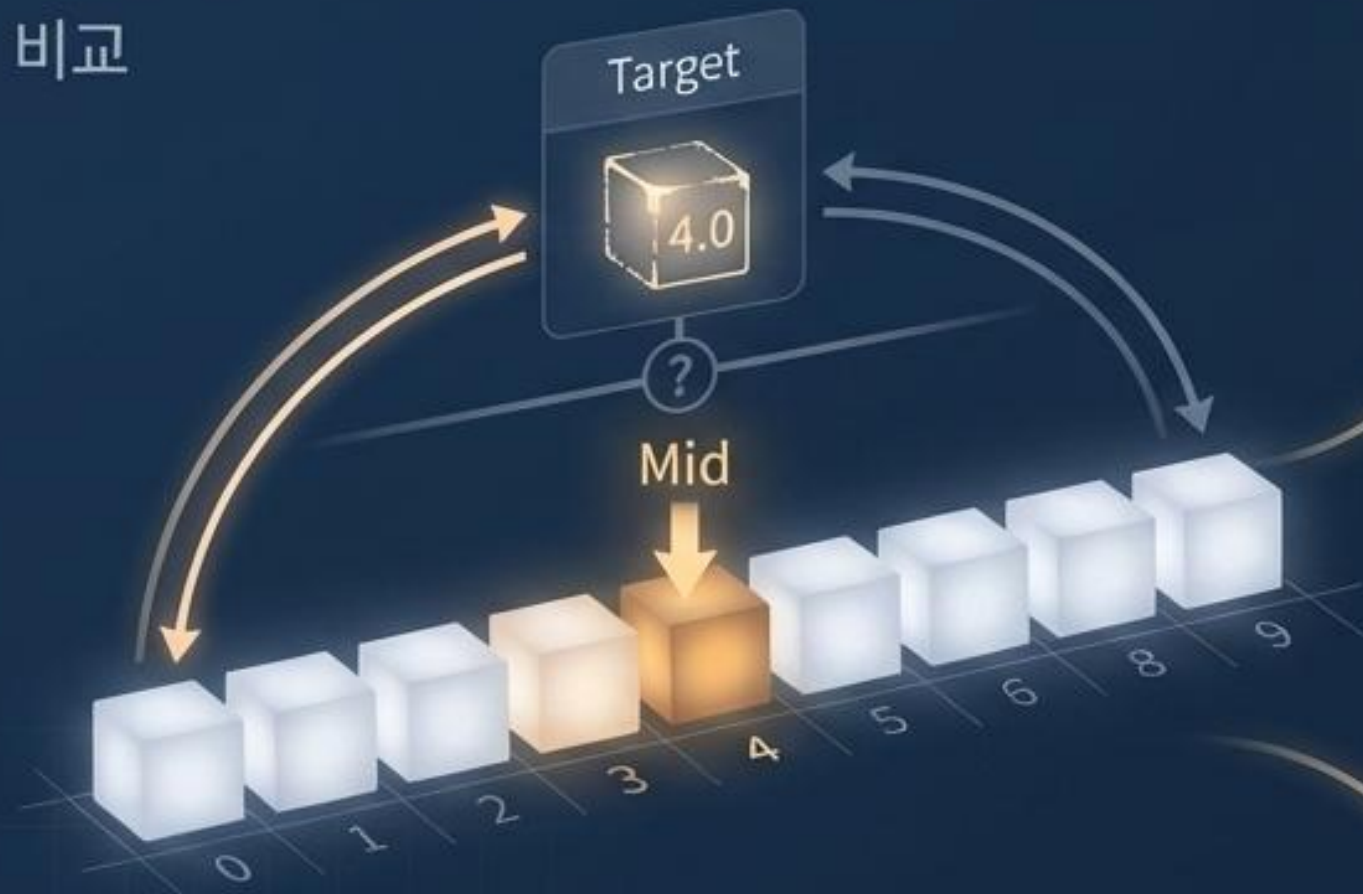


알고리즘의 원리는 단순합니다.

오직 이 3개의 포인터를 요리조리 움직이며 탐색 범위를 좁혀나가는 게임입니다.

# Action: 타겟이 중앙값(Mid)보다 큰가, 작은가?

비교



Target이 더 작을 때



Target이 더 클 때



찾고자 하는 값이 중간값보다 크다면, 왼쪽 절반은 더 이상 볼 필요가 없습니다. 과감하게 버리세요!



# 알고리즘을 파이썬 함수로 설계하기

2개의 재료를 입력받아 1개의 결과값을 밖으로 던져주는 (0, 0) 형태의 완벽한 사용자 정의 함수를 조립해 봅시다.



# 언제까지 탐색할 것인가? 멈춰야 할 때를 아는 브레이크

while문은 조건이 참인 동안 반복합니다.  $\text{left} \leq \text{right}$ 란 "아직 찾아볼 구간이 남아있다"는 의미입니다. 두 포인터가 역전되는 순간(데이터가 리스트에 없음), 루프는 종료됩니다.

```
left = 0  
right = len(arr) - 1
```

→ 

```
while left <= right:
```





# 방 번호(인덱스)는 반드시 정수여야 한다

파이썬의 정수 나누기 연산자 `//`가 이진 탐색 코드에서 절대적으로 필요한 이유입니다.

$$(1 + 4) / 2 = 2.5$$



에러! 소수점 방 번호는 존재하지 않습니다

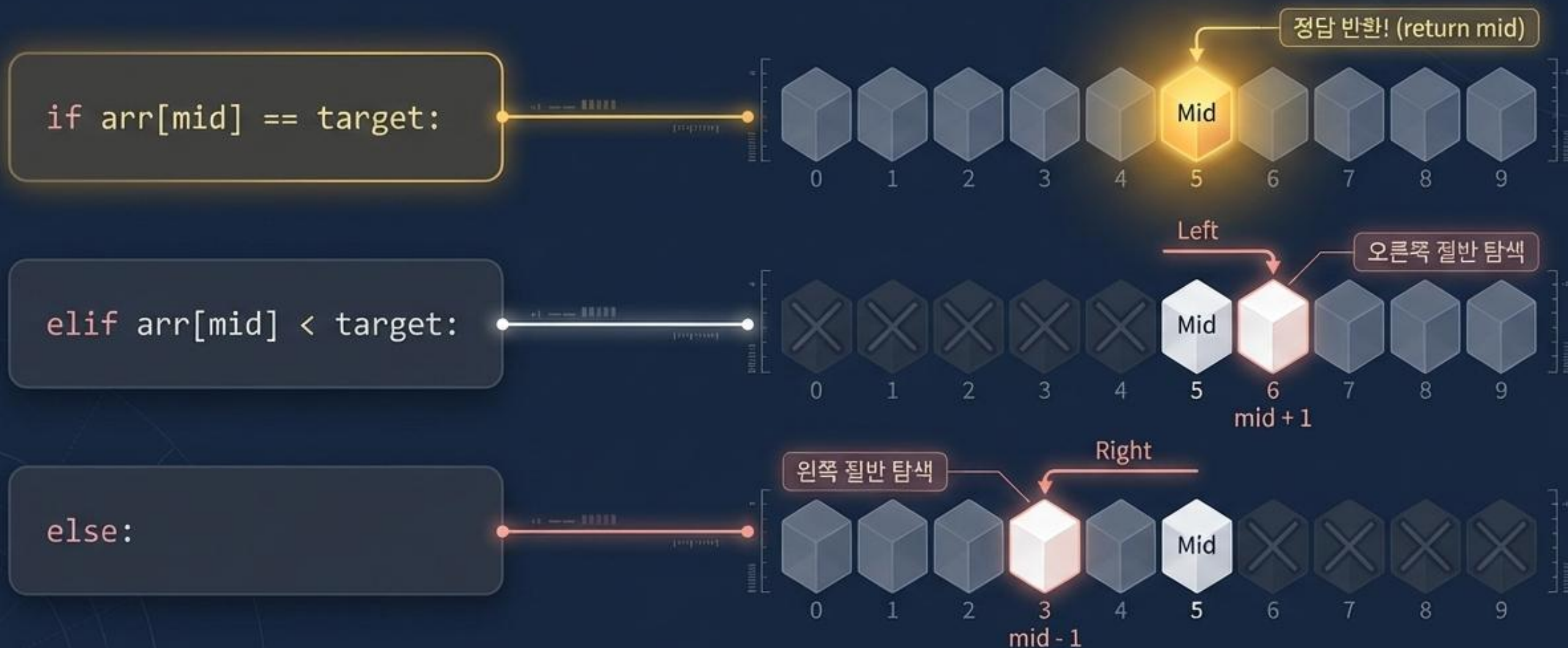
$$(1 + 4) // 2 = 2$$



성공! 소수점을 버리고  
정확한 인덱스를 찾습니다

# 조건문으로 탐색 범위 절반 날려버리기

단 3갈래의 조건 분기문이 방대한 데이터를 무자비하게 반으로 잘라내는 정밀 타격 조준경 역할을 합니다.





# 코드가 실행되는 찰나의 순간 (루프 1회차)

첫 번째 탐색. 단 번의 비교로 데이터의 절반(0~4 구간)이 완전히 탐색 대상에서 제외됩니다.

```
def binary_search(arr, target):  
    left = 0  
    right = len(arr) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1
```



# 타겟 발견 및 탐색 종료

단 2번의 루프만에 탐색 완료. 순차탐색이었다면 8번의 확인이 필요했을 작업입니다.

```
def array(target) {  
  let atpct = 7;  
  
  target = array[mid];  
  mid = mid;  
  if arr[mid] == target:  
    return mid  
  }  
  else {  
    return fightn;  
  }  
  
  return mid;  
}
```





# 데이터가 천만 개라면? (최악의 경우 비교)

| 데이터 개수      | 순차 탐색 (비교 횟수) | 이진 탐색 (비교 횟수) |
|-------------|---------------|---------------|
| 100개        | 100번          | 7번            |
| 10,000개     | 10,000번       | 14번           |
| 1,000,000개  | 1,000,000번    | 20번           |
| 10,000,000개 | 10,000,000번   | 24번           |

**천만 개의 데이터를 검사하는데 단 24번이면 충분합니다!**



# 실전 대결: '이효주' 학생 찾기

## 순차 탐색



0번 강수빈부터 하나씩 확인.  
결과: 비교 횟수 8번

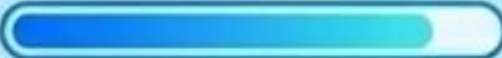
## 이진 탐색



중간 위치 '박민서' 확인 -> 아래 절반의 중간 '이효주' 확인  
결과: 비교 횟수 2번



# 한눈에 보는 탐색 알고리즘 비교

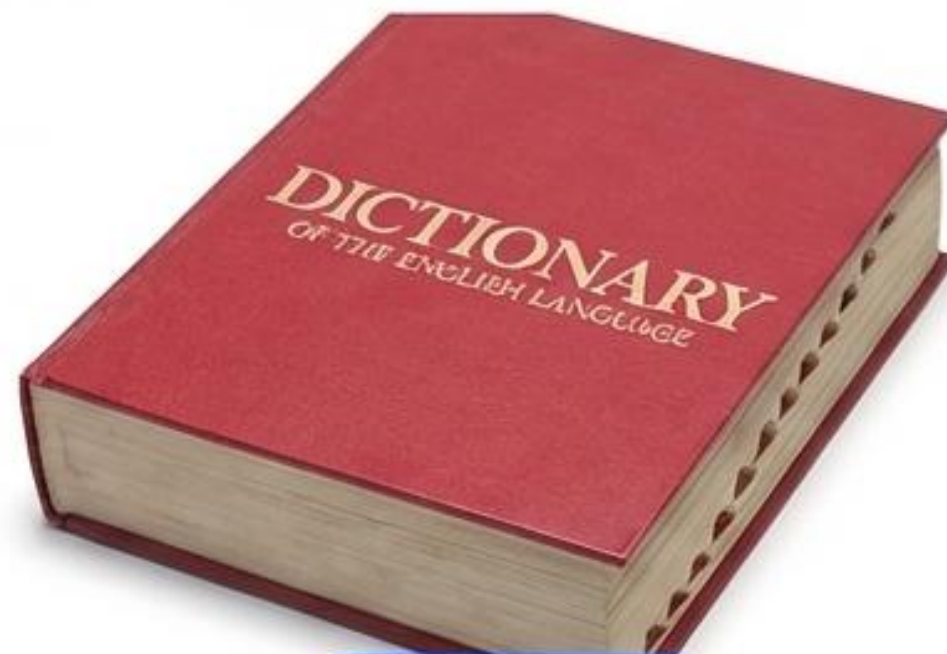
|            | 순차 탐색                                                                                                                                                                                                  | 이진 탐색                                                                                                                                                                                                      |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 탐색 방식      | 처음부터 하나씩<br>순서대로 검사<br>                                                                                             | 중간값을 기준으로<br>절반씩 버리며 탐색<br>                                                                                             |
| 사전 조건      |  정렬 불필요<br>(아무렇게나 섞여 있어도 OK)                                                                                      |  반드시 오름차순/내림차순<br>정렬되어야 함                                                                                             |
| 대규모 데이터 효율 | 비효율적<br>(데이터가 많을수록 느려짐)<br>  | 매우 효율적<br>(데이터가 방대해도 엄청 빠름)<br>  |

# 일상 속의 탐색 알고리즘



순차 탐색

도서관에서 책 찾기: 책들이 순서대로 꽂  
꽂혀있지 않다면? 1번 책장부터 하나씩 다  
찾아야 합니다.

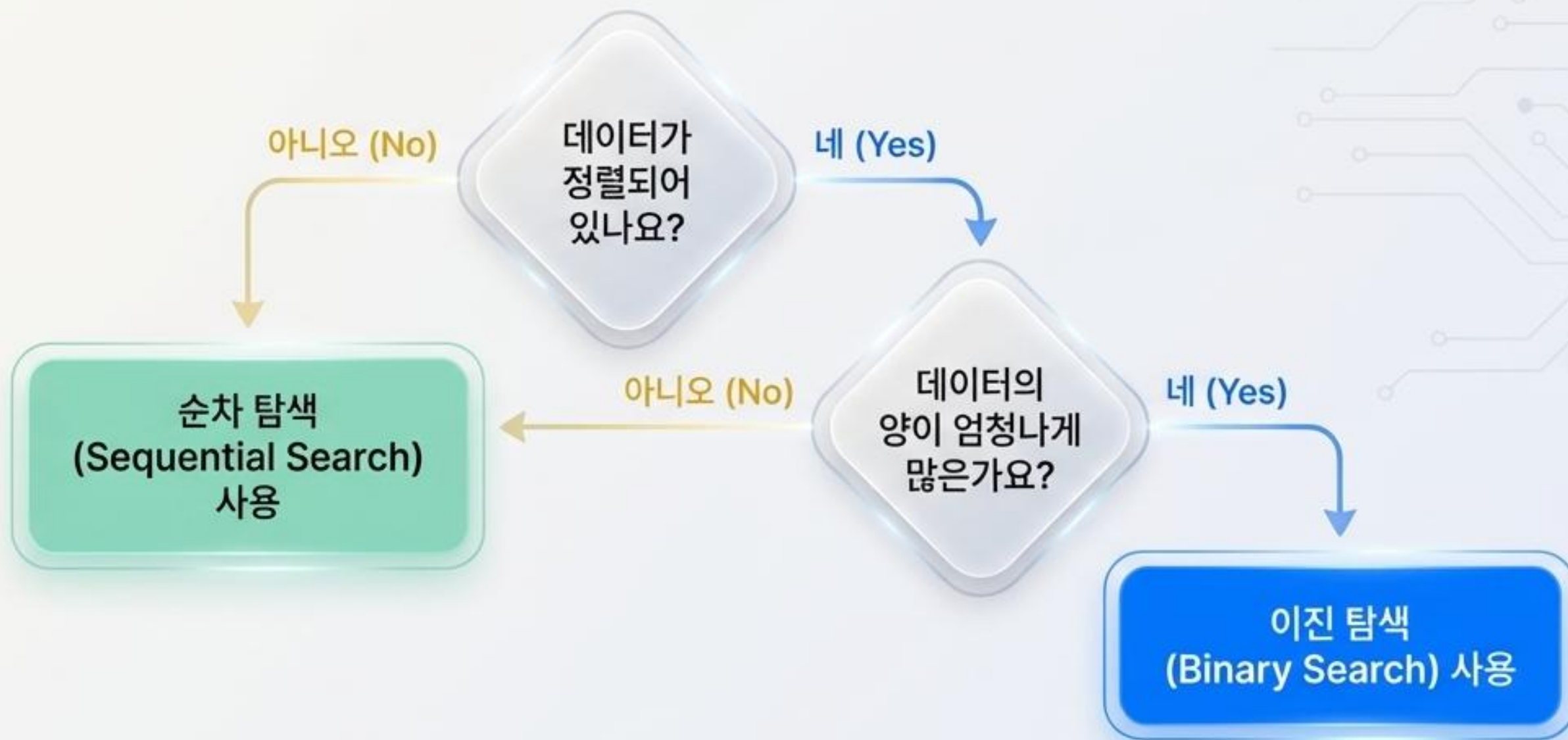


이진 탐색

사전에서 단어 찾기: 'Algorithm'을 찾을 때,  
중간을 펼쳐보고 알파벳 순서에 따라 남은 절반  
을 탐색합니다. (정렬되어 있기 때문에 가능!)



# 개발자의 선택: 어떤 알고리즘을 써야 할까?

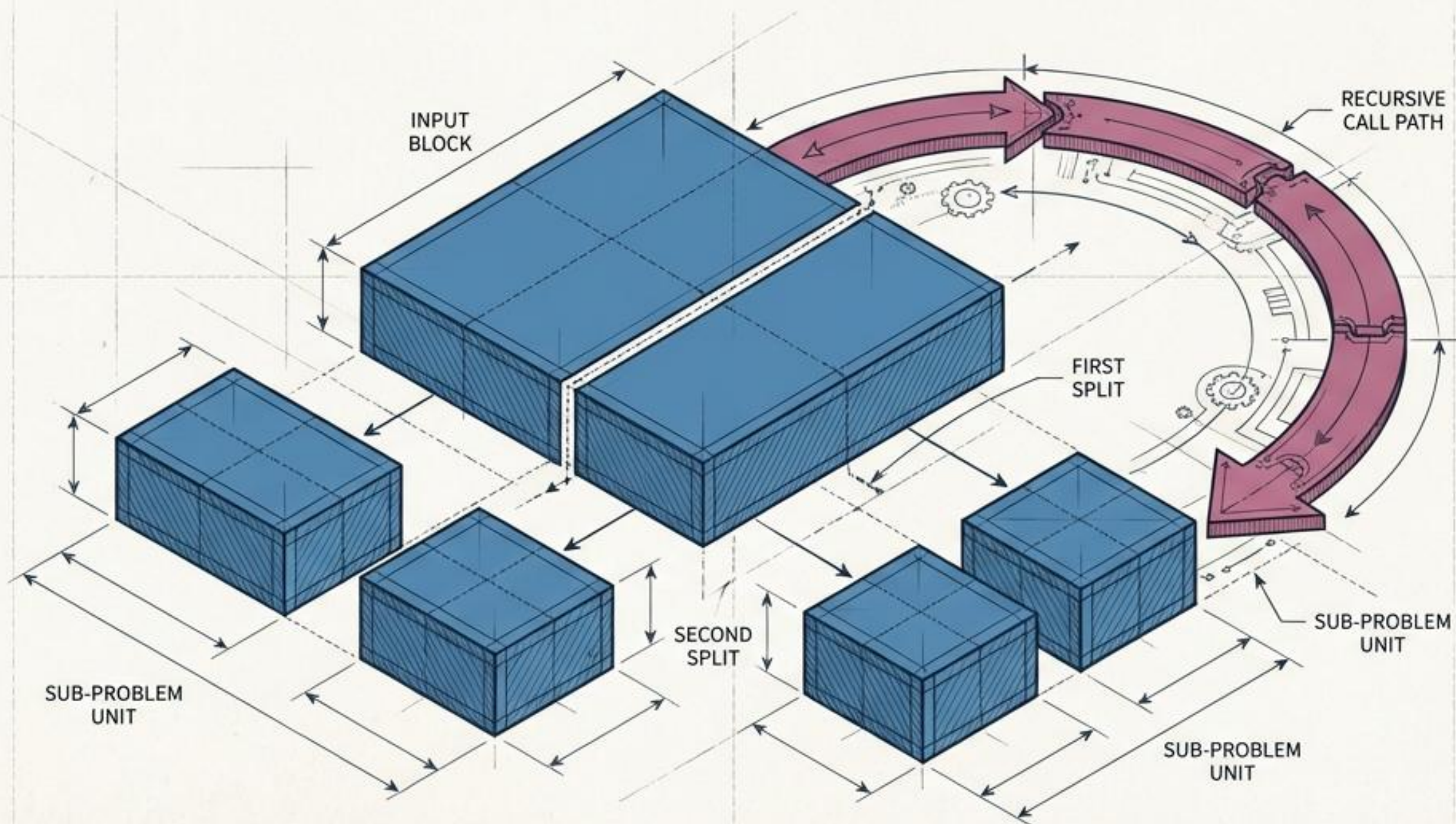


탐색 알고리즘은 데이터의 특성(정렬 여부, 크기)에 따라 완벽한 도구를 선택하는 것이 핵심입니다.



# 데이터의 바다를 탐험하는 분할 정복의 미학

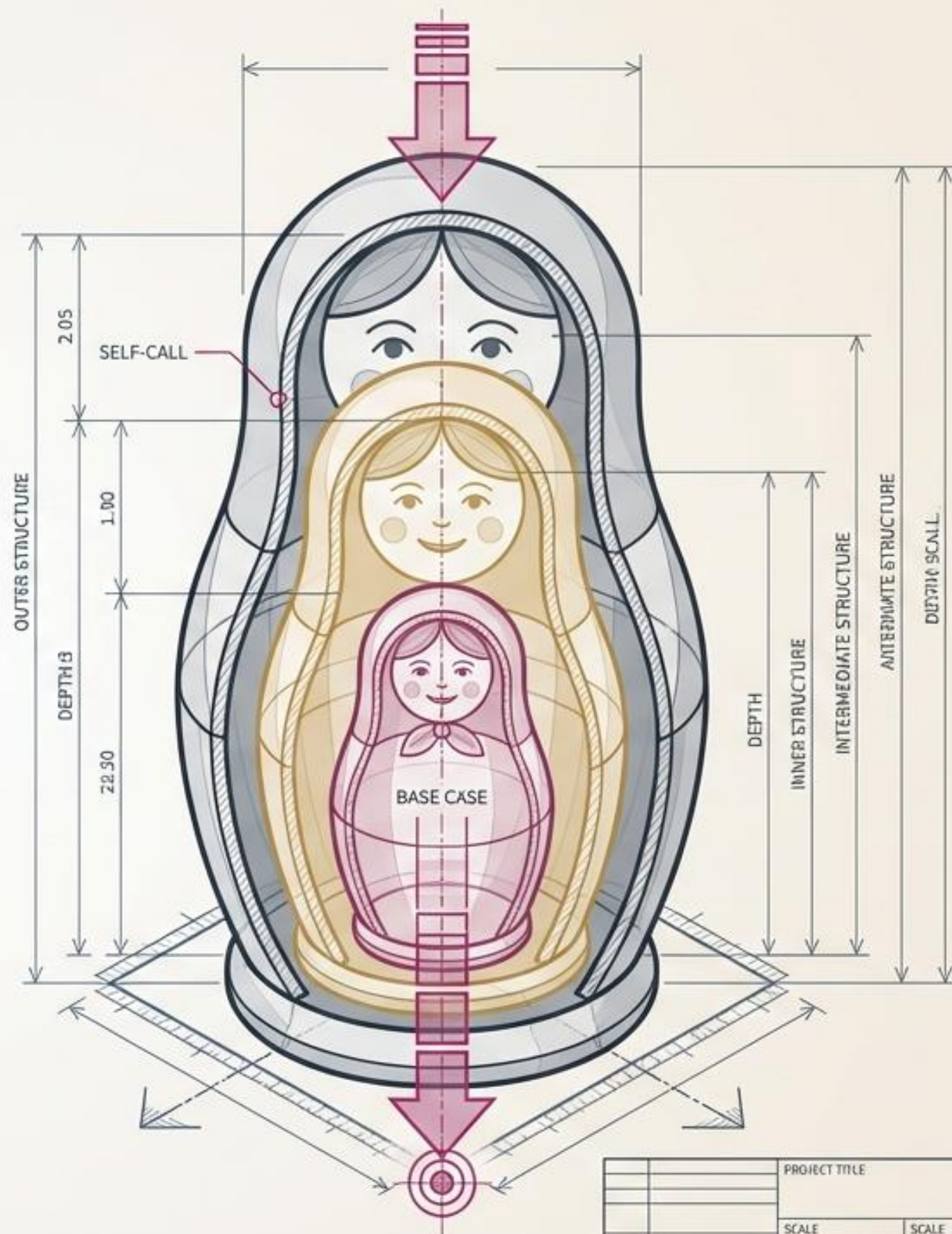
자기 자신을 호출하는 마법, 재귀함수로 구현하는 이진 탐색의 완벽한 논리 구조.





## 재귀(Recursion): 거울 속의 거울을 들여다보는 논리

재귀함수는 완전히 동일한 패턴의 더 작은 문제를 해결하기 위해 '자기 자신'을 다시 호출하는 함수입니다. 단순한 코딩기법이 아니라, 복잡한 현상을 가장 작은 단위로 쪼개어 바라보는 사고방식입니다.

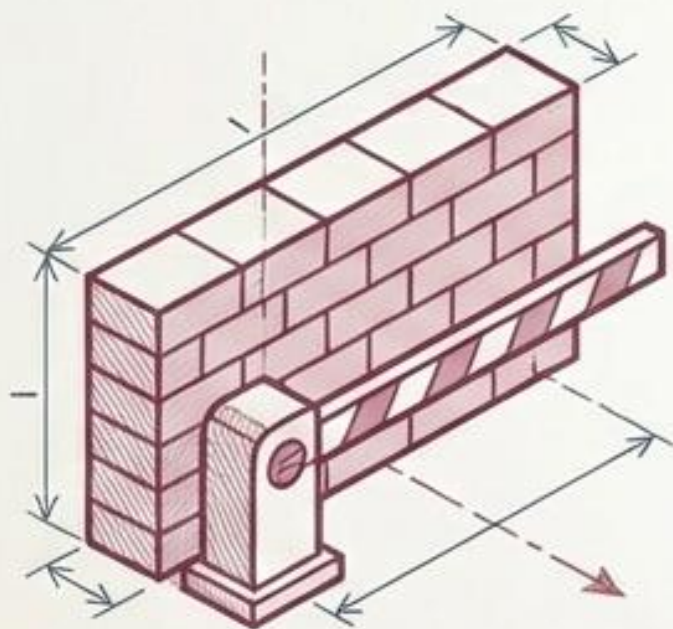




# 무한 루프를 막는 재귀함수의 두 가지 핵심 기동

모든 재귀함수는 무한히 빠져드는 것을 방지하기 위해 두 가지 필수 요소를 가져야 합니다.  
이 두 가지가 결합될 때 비로소 재귀는 '작동'합니다.

## 종료 조건 (Base Case)



더 이상 쪼갤 수 없을 때  
함수를 멈추는 명확한 기준.

## 재귀 호출 (Recursive Step)



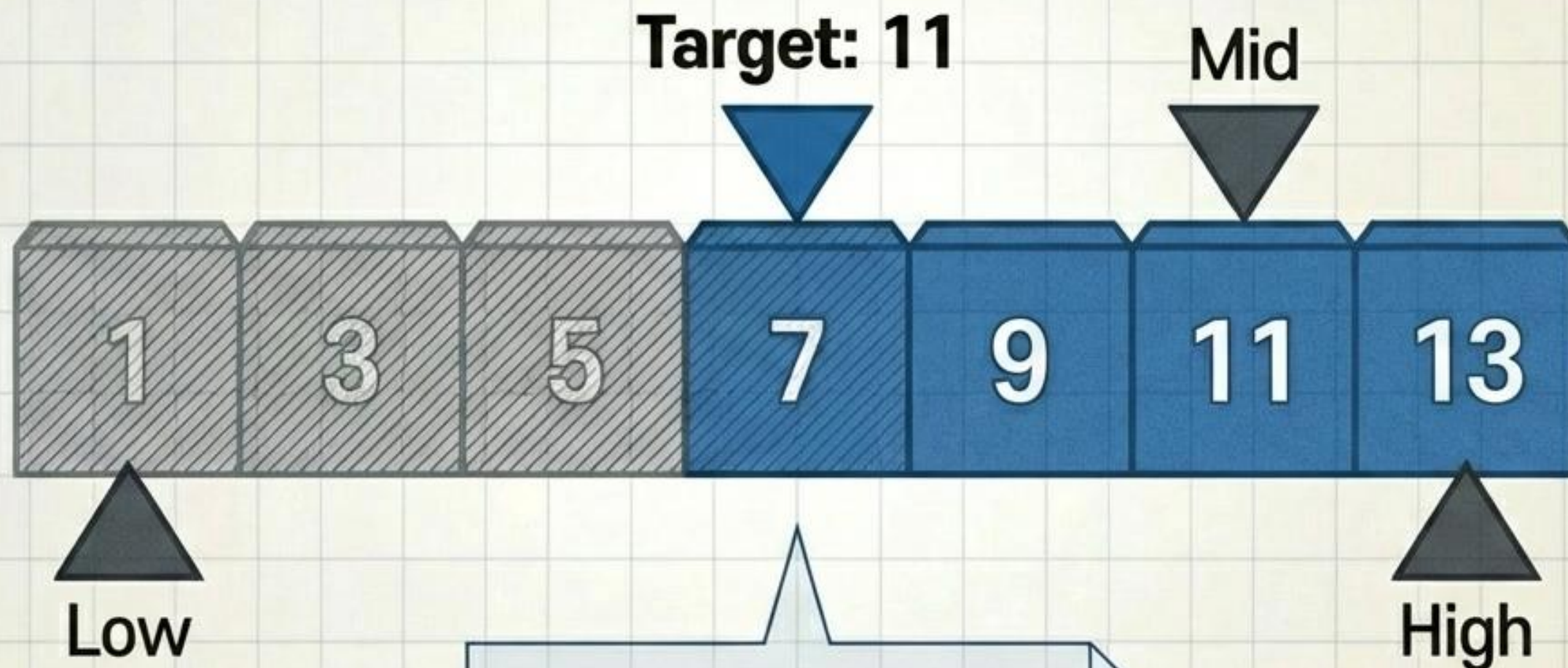
문제를 한 단계 더 작게 만들어  
자기 자신을 다시 부르는 과정.



```
def binary_search(arr, target, low, high):  
    if low > high:  
        return -1  
    mid = (low + high) // 2  
  
    if arr[mid] == target:    # 1. 중간값이 찾으려는 값과 일치하는 경우  
        return mid  
  
    elif arr[mid] > target:   # 2. 중간값이 타겟보다 큰 경우 -> 왼쪽 구간 탐색  
        return binary_search(arr, target, low, mid - 1)  
  
    else:                    # 3. 중간값이 타겟보다 작은 경우 -> 오른쪽 구간 탐색  
        return binary_search(arr, target, mid + 1, high)  
  
data = [ 2,5,6,8,9,11,15,18,19,22,24,25 ]  
result = binary_search(data, value, 0, len(data) - 1)
```

## 재귀적 이진 탐색의 첫 번째 단계: 중앙값 비교

배열의 정중앙을 확인합니다. 우리가 찾는 값(Target)이 중앙값보다 크다면, 왼쪽 절반은 더 이상 탐색할 필요가 없습니다. 이제 오른쪽 절반을 대상으로 '자기 자신'을 호출합니다.



Target(11) > Mid(7)

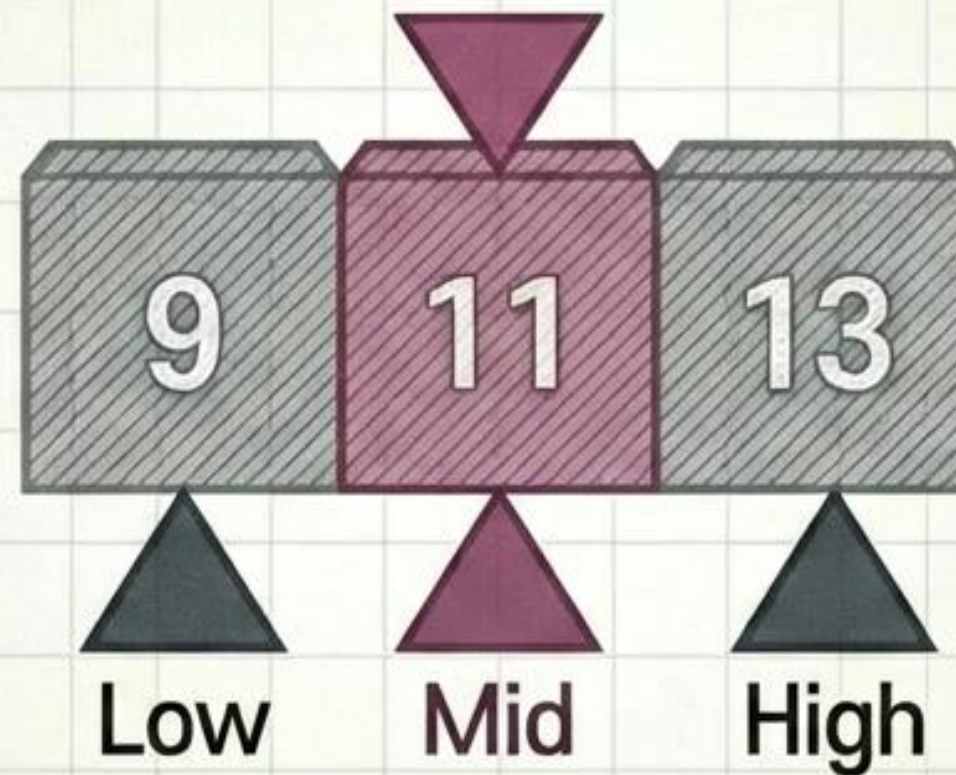


|  |  |               |
|--|--|---------------|
|  |  | PROJECT TITLE |
|  |  |               |
|  |  |               |
|  |  | SCALE         |
|  |  | SCALE DATE    |



## 새로운 범위에서의 재귀 호출과 탐색 종료

업데이트된 새로운 경계( $\text{Low} = \text{Mid} + 1$ )를 가지고 함수가 재귀적으로 실행됩니다.  
중앙값이 우리가 찾는 값과 일치하는 순간, 탐색은 즉시 종료(Base Case)됩니다.



Target(11) == Mid(11)



## 정답 발견! (Base Case)

[illegible]



# 인간의 논리를 그대로 닮은 파이썬 코드 구조

복잡한 반복문 없이, 오직 순수한 논리적 조건만으로 이루어진  
깔끔한 코드 아키텍처를 확인해 보세요.

```
if low > high:  
    return -1
```

종료 조건 1: 탐색 실패  
(Base Case)

```
if arr[mid] == target:  
    return mid
```

종료 조건 2: 탐색 성공  
(Base Case)

```
return binary_search_recursive(...)
```

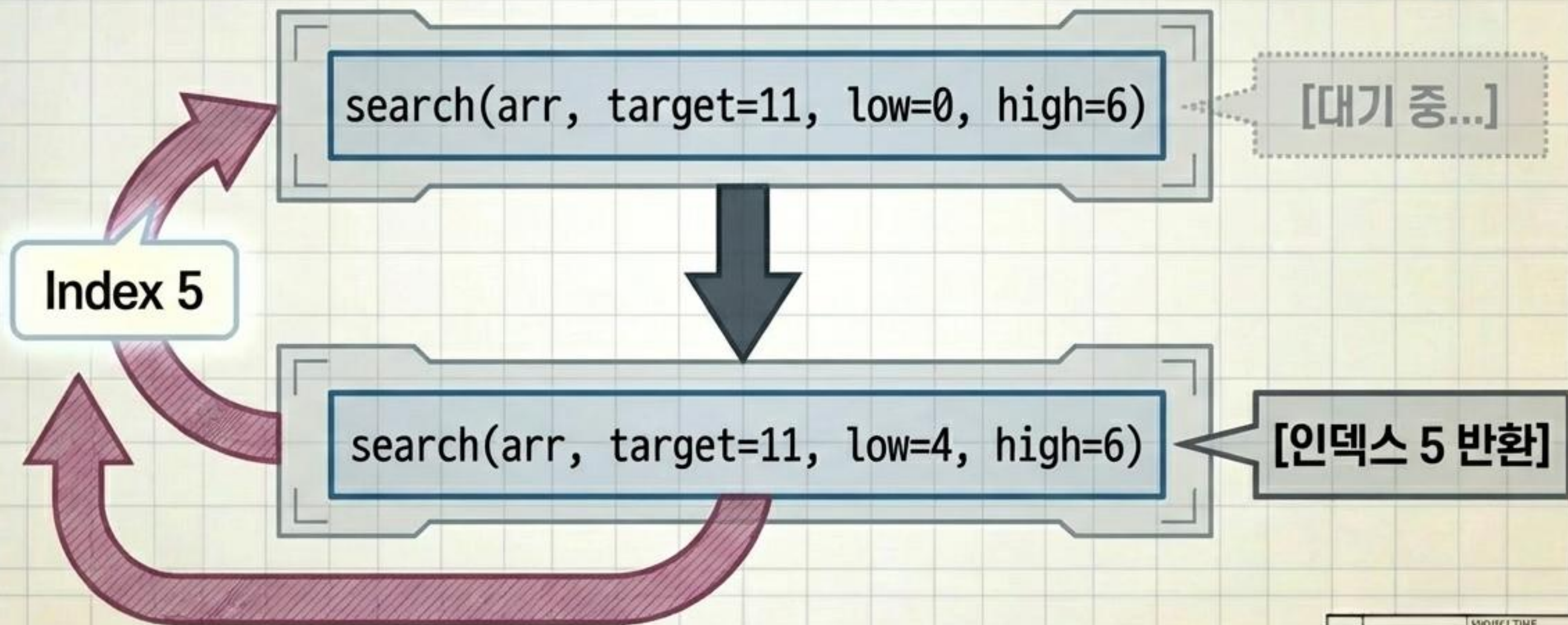
재귀 호출: 범위를 좁혀  
자기 자신을 다시 호출





# 컴퓨터는 정답을 어떻게 위로 전달하는가?

정답을 찾았다고 끝나는 것이 아닙니다. 메모리 깊은 곳에서 찾은 인덱스 값은 콜 스택을 타고 역순으로 호출자에게 연쇄적으로 전달되어야 비로소 최종 결과가 됩니다.





# 반복문(While)과 재귀함수(Recursive)의 구조적 차이

두 방식 모두 동일한 이진 탐색을 수행하지만, 코드를 설계하는 철학과 메모리를 사용하는 방식에서 명확한 차이를 보입니다.

|                         | 반복문 방식 (Iterative)            | 재귀함수 방식 (Recursive)                      |
|-------------------------|-------------------------------|------------------------------------------|
| 논리 전개 /<br>Logical Flow | 상태(변수)를 갱신하며<br>루프를 도는 구조     | 더 작은 문제로 쪼개어<br>자기 자신에게 위임하는 구조          |
| 메모리 효율 /<br>Memory      | 메모리 사용량이 적음<br>(콜 스택 오버헤드 없음) | 재귀 깊이가 깊어질수록<br>콜 스택 메모리 사용량 증가          |
| 코드 가독성 /<br>Readability | 루프 조건과 상태 변화를<br>추적해야 함       | 압도적으로 우수함. 수학적 정의와<br>인간의 논리를 코드에 그대로 매핑 |





# 1-3 탐색 알고리즘 핵심 요약



방대한 데이터 시대,  
빠른 탐색 알고리즘은  
현대 기술의 필수 엔진입니다.



순차 탐색은 유연하지만  
데이터가 많아질수록 느려지는  
 $O(n)$ 의 한계가 있습니다.



이진 탐색은 절반씩  
탐색 범위를 줄여 압도적으로  
빠르지만, 반드시 '정렬'이라는  
대가가 필요합니다.